Lecture 1: Dijkstra's Shortest Paths Algorithm

Dr. George Mertzios

george.mertzios@durham.ac.uk

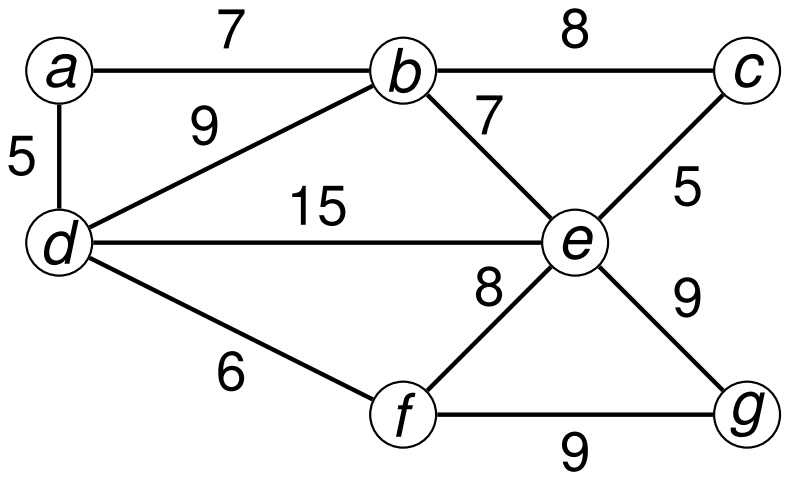
Room 2066, MCS Building
Tel: 42429

Representing graphs with edge weights

To give an adjacency matrix representation of a graph with edge weights, we use a square matrix and label the rows and columns with the vertices.

The entry in row i and column j is:

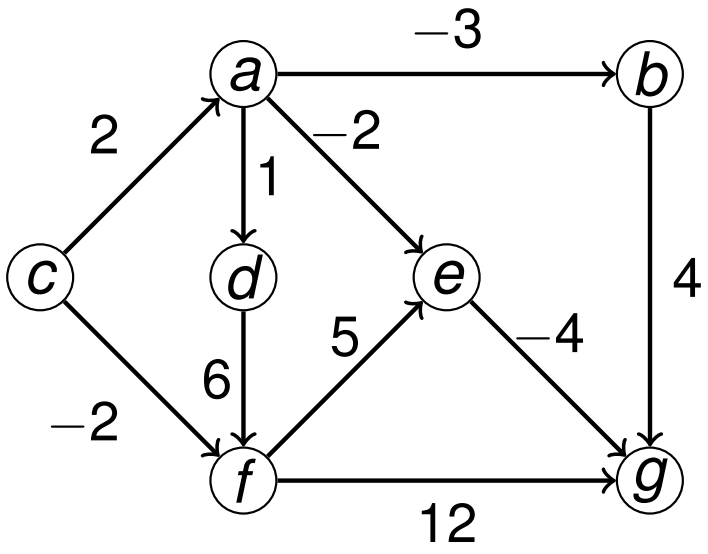
- w if there is an edge of weight w from i and j,
- 0 if there is no edge.



$$\begin{pmatrix} 0 & 7 & 0 & 5 & 0 & 0 & 0 \\ 7 & 0 & 8 & 9 & 7 & 0 & 0 \\ 0 & 8 & 0 & 0 & 5 & 0 & 0 \\ 5 & 9 & 0 & 0 & 15 & 6 & 0 \\ 0 & 7 & 5 & 15 & 0 & 8 & 9 \\ 0 & 0 & 0 & 6 & 8 & 0 & 9 \\ 0 & 0 & 0 & 0 & 9 & 9 & 0 \end{pmatrix}$$

Shortest Paths

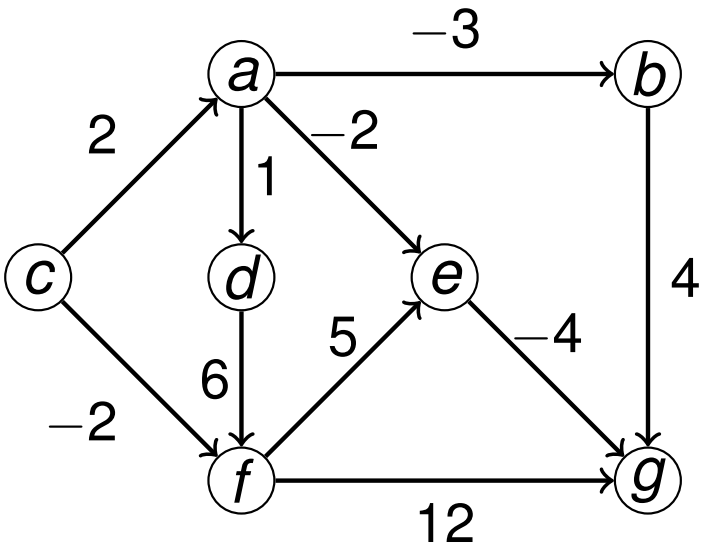
The weight (or, informally, the length) of a path from a vertex u to a vertex v is the sum of the weights of the edges in the path. Note that a weight may be negative.



There are paths of lengths 3, -1 and 21 (and others) between c and g.

Shortest Paths

The weight (or, informally, the length) of a path from a vertex u to a vertex v is the sum of the weights of the edges in the path. Note that a weight may be negative.

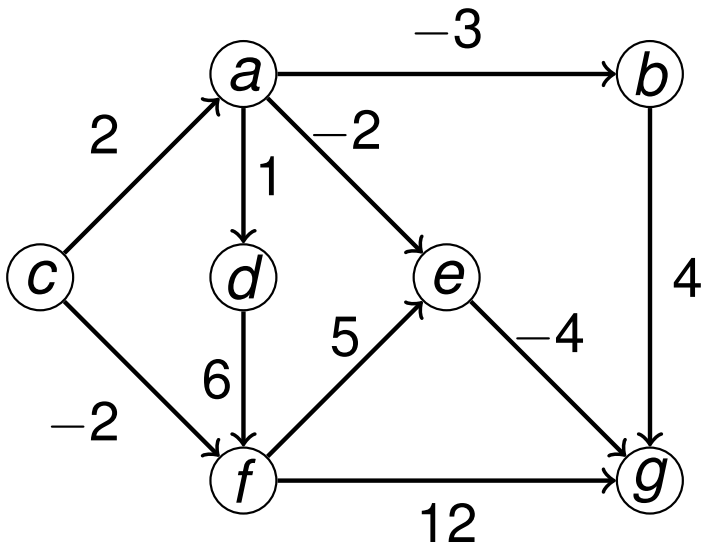


There are paths of lengths 3, -1 and 21 (and others) between c and g.

The shortest path (i.e. with least weight) between u and v is denoted $\delta(u, v)$. If there is no path, then $\delta(u, v) = \infty$.

Shortest Paths

The weight (or, informally, the length) of a path from a vertex u to a vertex v is the sum of the weights of the edges in the path. Note that a weight may be negative.



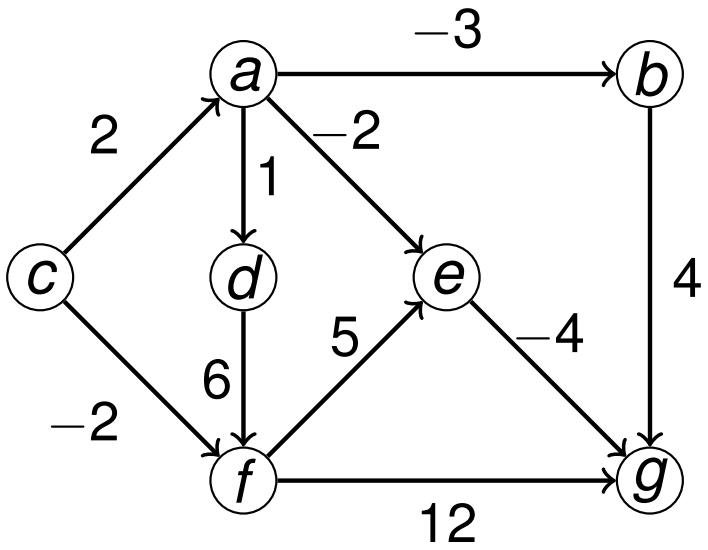
There are paths of lengths 3, -1 and 21 (and others) between c and g.

The shortest path (i.e. with least weight) between u and v is denoted $\delta(u, v)$. If there is no path, then $\delta(u, v) = \infty$.

Above, $\delta(c, g) =$???

Shortest Paths

The weight (or, informally, the length) of a path from a vertex u to a vertex v is the sum of the weights of the edges in the path. Note that a weight may be negative.



There are paths of lengths 3, -1 and 21 (and others) between c and g.

The shortest path (i.e. with least weight) between u and v is denoted $\delta(u, v)$. If there is no path, then $\delta(u, v) = \infty$.

Above, $\delta(c, g) = -4$ (the path $c - a - e - g$)

Can a shortest path contain a cycle?

A directed cycle is:

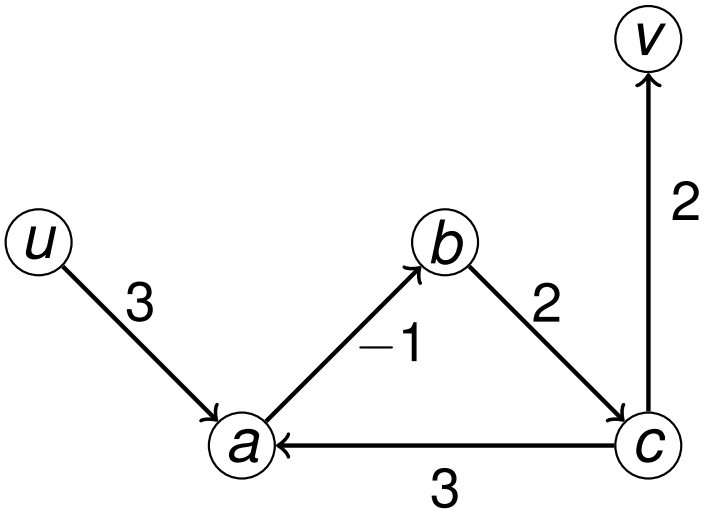
- positive, if its edge weights sum up to a positive number,
- negative, if its edge weights sum up to a negative number.

Can a shortest path contain a cycle?

A directed cycle is:

- positive, if its edge weights sum up to a positive number,
- negative, if its edge weights sum up to a negative number.

- If there is a positive cycle in the graph, it will not be contained in any shortest path between u and v.



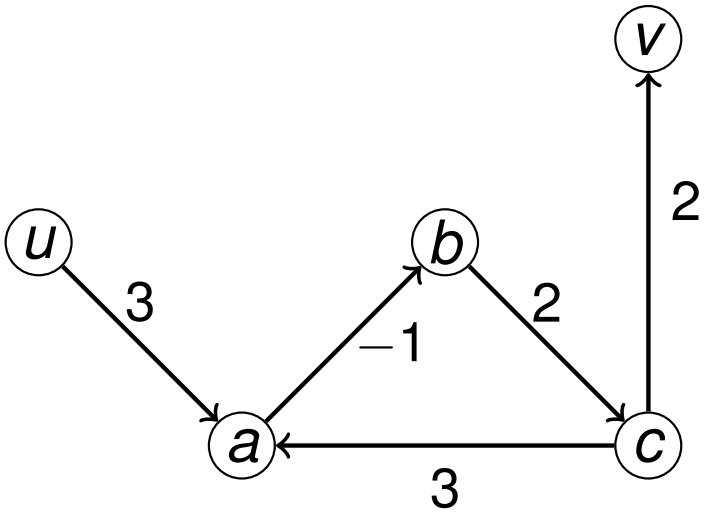
Can a shortest path contain a cycle?

A directed cycle is:

- positive, if its edge weights sum up to a positive number,
- negative, if its edge weights sum up to a negative number.

- If there is a positive cycle in the graph, it will not be contained in any shortest path between u and v.

$\Longrightarrow$ we can assume that the shortest paths we find contain no positive cycles.

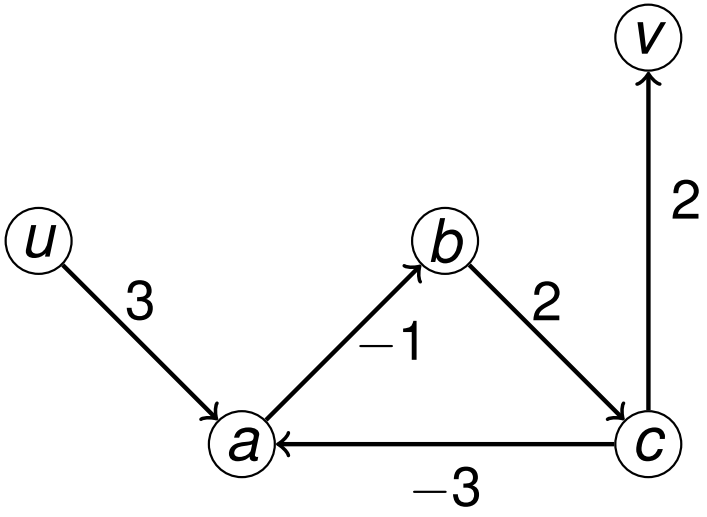


Can a shortest path contain a cycle?

A directed cycle is:

- positive, if its edge weights sum up to a positive number,
- negative, if its edge weights sum up to a negative number.

- If there is a negative cycle between u and v, then $\delta(u, v) =$???.

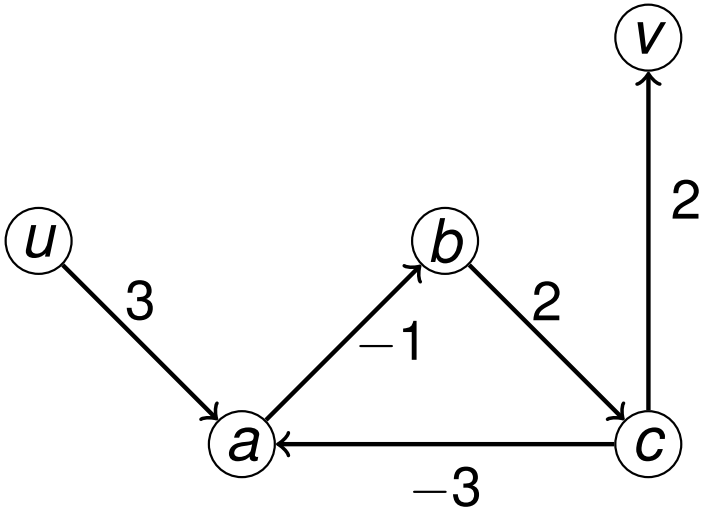


Can a shortest path contain a cycle?

A directed cycle is:

- positive, if its edge weights sum up to a positive number,
- negative, if its edge weights sum up to a negative number.

- If there is a negative cycle between u and v, then $\delta(u, v) = -\infty$.



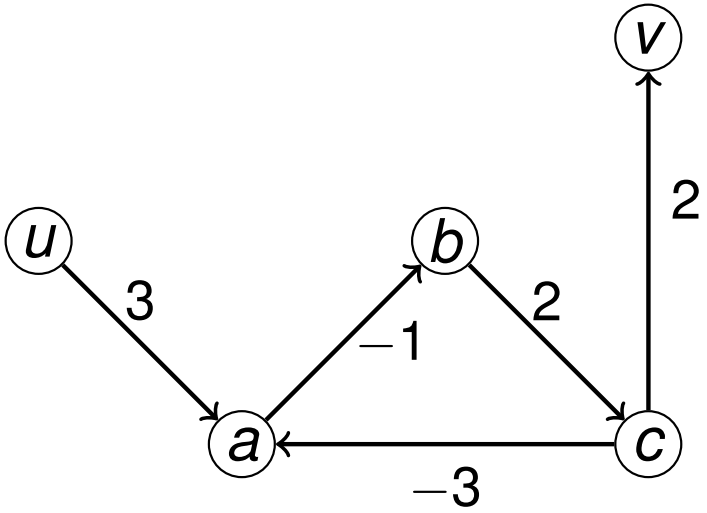
Can a shortest path contain a cycle?

A directed cycle is:

- positive, if its edge weights sum up to a positive number,
- negative, if its edge weights sum up to a negative number.

- If there is a negative cycle between u and v, then $\delta(u, v) = -\infty$.

$\Longrightarrow$ we shall assume that the graphs we consider do not contain negative cycles.



Single-Source Shortest Paths

- Aim: to describe an algorithm that solves the Single-Source Shortest Paths problem,

Single-Source Shortest Paths

- Aim: to describe an algorithm that solves the Single-Source Shortest Paths problem,
  i.e. an algorithm that finds the shortest path
  from a specific source vertex s to all other vertices in the graph.

Single-Source Shortest Paths

- Aim: to describe an algorithm that solves the Single-Source Shortest Paths problem,
  i.e. an algorithm that finds the shortest path
  from a specific source vertex s to all other vertices in the graph.
- This is a generalization of Breadth-First Search
  (BFS is the special case where each weight is 1).

Single-Source Shortest Paths

- Aim: to describe an algorithm that solves the Single-Source Shortest Paths problem,
  i.e. an algorithm that finds the shortest path
  from a specific source vertex s to all other vertices in the graph.
- This is a generalization of Breadth-First Search
  (BFS is the special case where each weight is 1).
- So the output of the algorithm should be two arrays d, π where for each vertex v:
  - $d(v) = \delta(s, v)$,
  - $\pi(v)$ is the predecessor of v.

Relaxation: a useful technique

- Assume that the weight on every edge is non-negative (such as a road-map).
- We do not directly compute the entry $d(v) = \delta(s, v)$ (in contrast to BFS)
- Instead, at every step, $d(v)$ is an estimate for $\delta(s, v)$:
  - initially $d(v) = \infty$, and it always remains $d(v) \geq \delta(s, v)$;
  - $d(v)$ is updated (i.e. it decreases), as shorter paths are found;
  - at the end of the algorithm we have $d(v) = \delta(s, v)$.

Relaxation: a useful technique

- Assume that the weight on every edge is non-negative (such as a road-map).
- We do not directly compute the entry $d(v) = \delta(s, v)$ (in contrast to BFS)
- Instead, at every step, $d(v)$ is an estimate for $\delta(s, v)$:
  - initially $d(v) = \infty$, and it always remains $d(v) \geq \delta(s, v)$;
  - $d(v)$ is updated (i.e. it decreases), as shorter paths are found;
  - at the end of the algorithm we have $d(v) = \delta(s, v)$.

```
INITIALIZE-SINGLE-SOURCE(G, s)
for each vertex v ∈ V(G) do
    d(v) ← ∞
    π(v) ← NIL
d(s) ← 0
```

Relaxation: a useful technique

The process of relaxing an edge (u, v):

- test whether we can improve the shortest path from s to v that we found so far, by going through u
- if yes, then update $d(v)$ and $\pi(v)$:
  - decrease the estimate $d(v)$
  - update the predecessor $\pi(v)$ to u

Relaxation: a useful technique

The process of relaxing an edge (u, v):

- test whether we can improve the shortest path from s to v that we found so far, by going through u
- if yes, then update $d(v)$ and $\pi(v)$:
  - decrease the estimate $d(v)$
  - update the predecessor $\pi(v)$ to u
- The algorithm first calls INITIALIZE-SINGLE-SOURCE and then it repeatedly relaxes the appropriate edges (according to the weight function w).

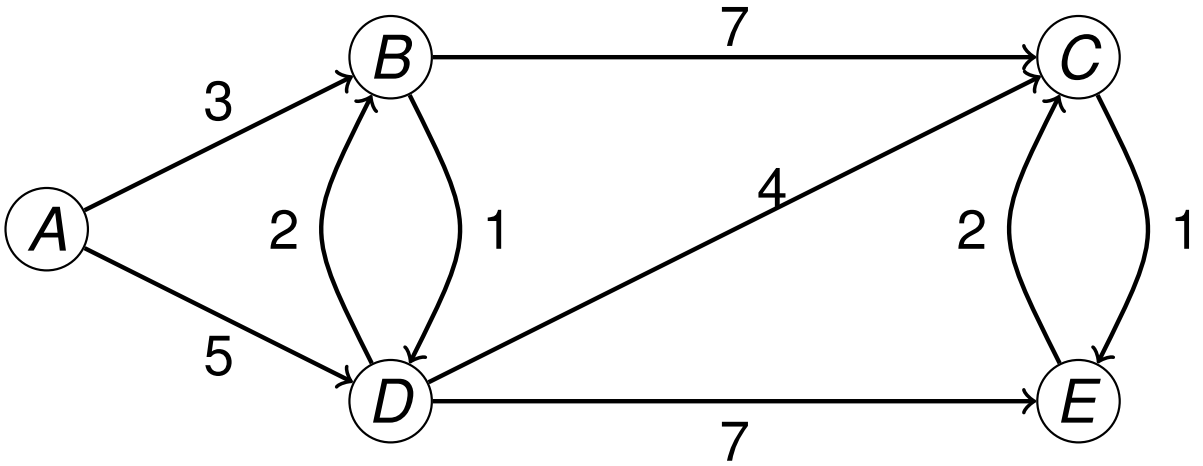
Relaxation: a useful technique

The process of relaxing an edge (u, v):

- test whether we can improve the shortest path from s to v that we found so far, by going through u
- if yes, then update $d(v)$ and $\pi(v)$:
  - decrease the estimate $d(v)$
  - update the predecessor $\pi(v)$ to u
- The algorithm first calls INITIALIZE-SINGLE-SOURCE and then it repeatedly relaxes the appropriate edges (according to the weight function w).

```
RELAX(u, v, w)
if d(v) > d(u) + w(u, v) then
    d(v) ← d(u) + w(u, v)
    π(v) ← u
```

Dijkstra's Algorithm

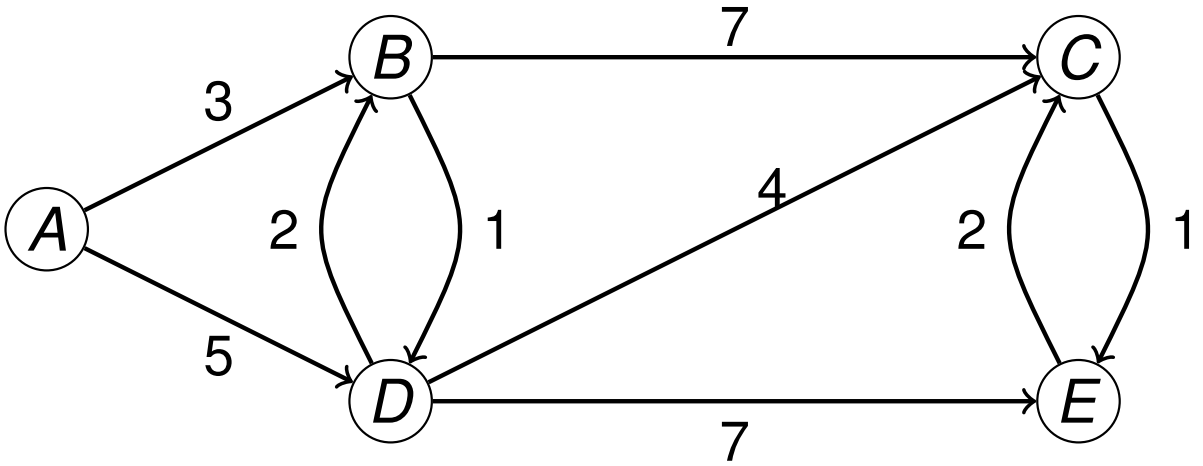


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \emptyset \qquad Q = \{A, B, C, D, E\}$

	A	B	C	D	E
d	0	∞	∞	∞	∞
π	$-$				

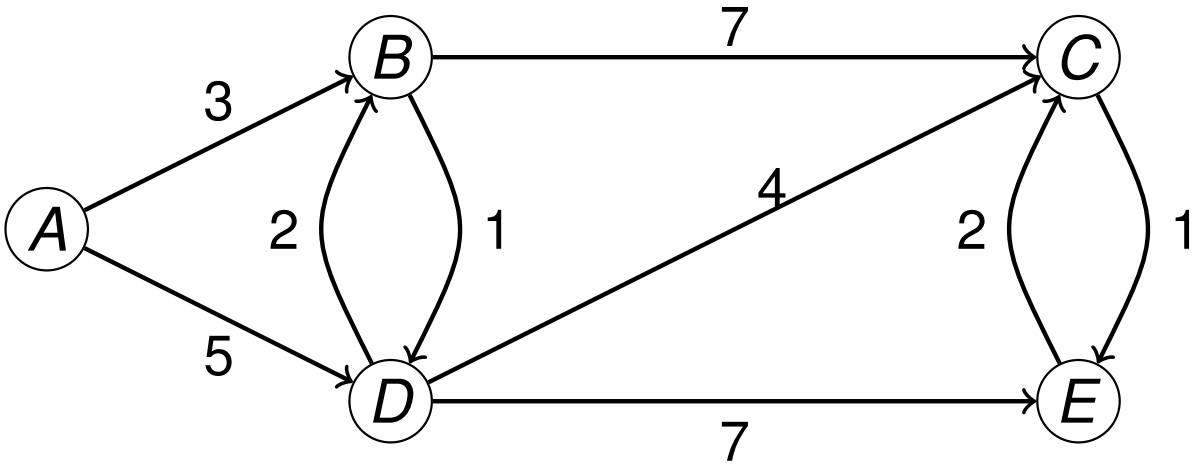


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A\}$ $\quad Q = \{B, C, D, E\}$

	A	B	C	D	E
d	0	∞	∞	∞	∞
π	$-$				

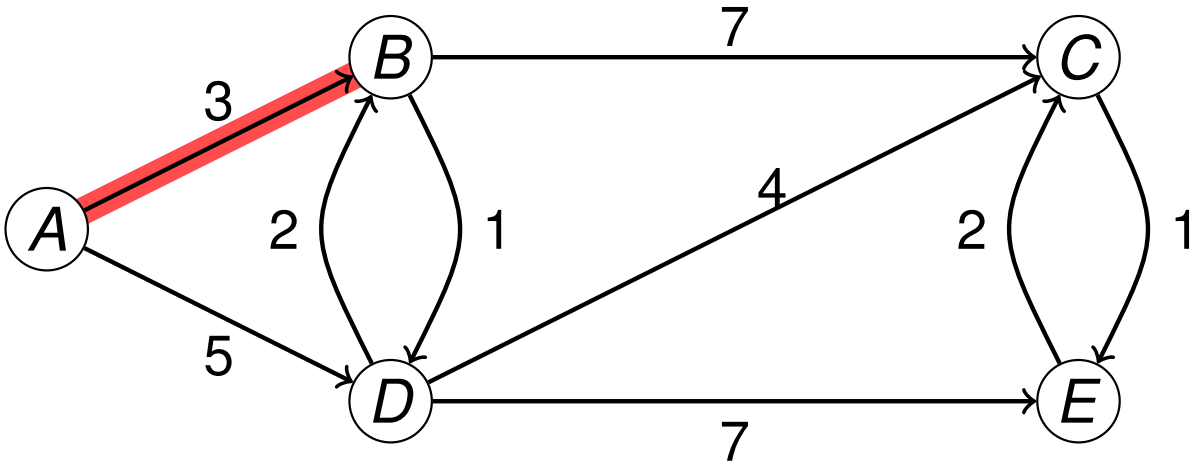


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A\}$ $\quad Q = \{B, C, D, E\}$

	A	B	C	D	E
d	0	∞	∞	∞	∞
π	$-$				

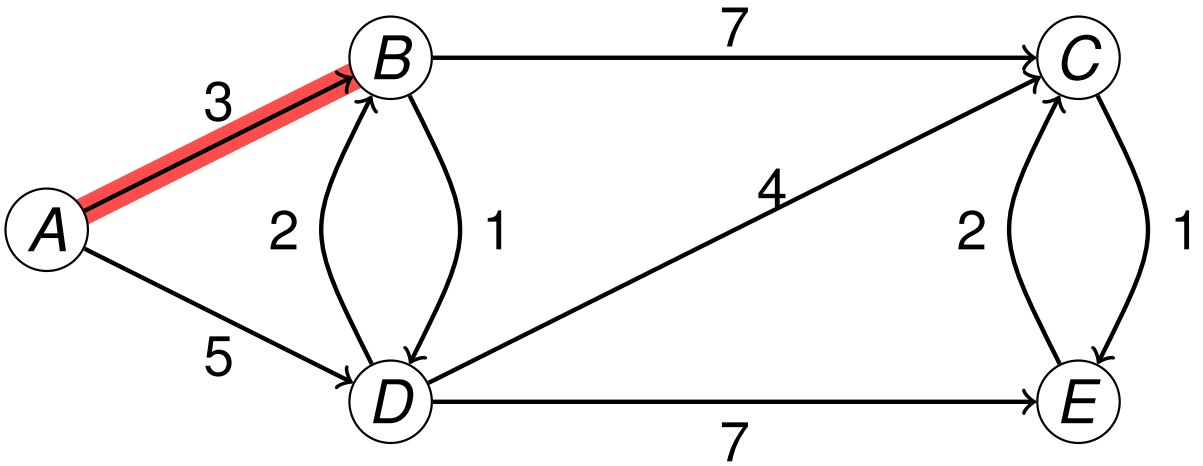


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A\}$ $\quad Q = \{B, C, D, E\}$

	A	B	C	D	E
d	0	3	∞	∞	∞
π	–	A			

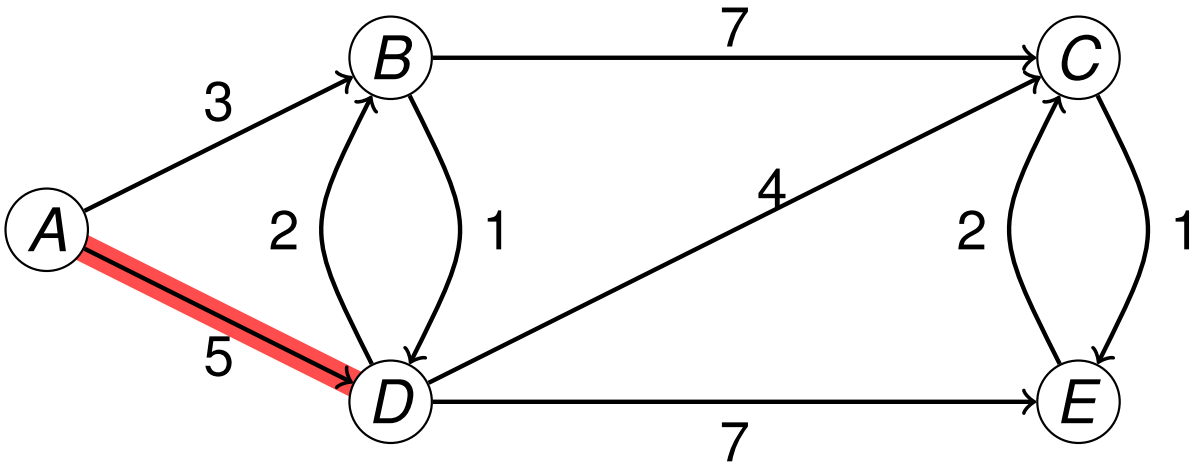


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A\}$ $\quad Q = \{B, C, D, E\}$

	A	B	C	D	E
d	0	3	∞	∞	∞
π	–	A			

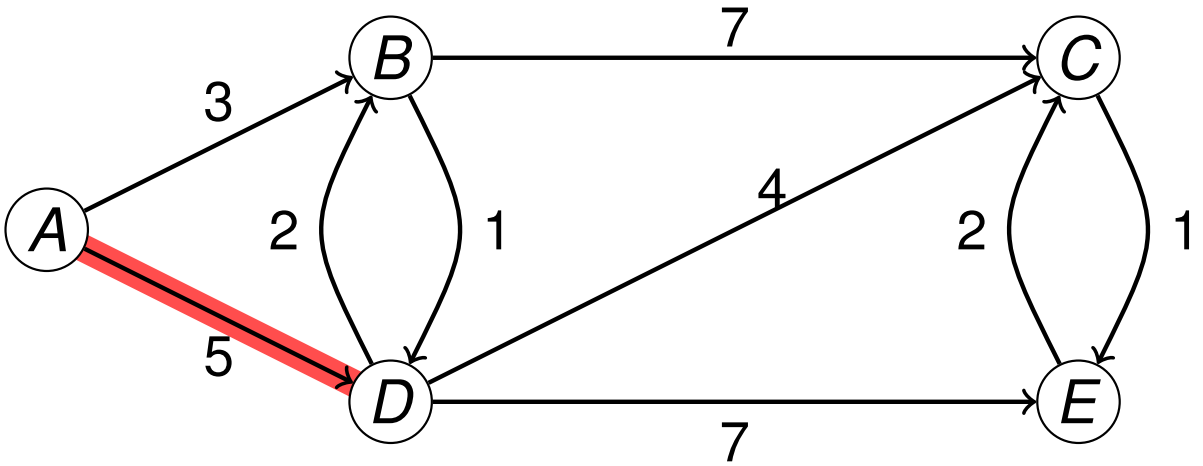


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A\}$ $\quad Q = \{B, C, D, E\}$

	A	B	C	D	E
d	0	3	∞	5	∞
π	–	A		A	

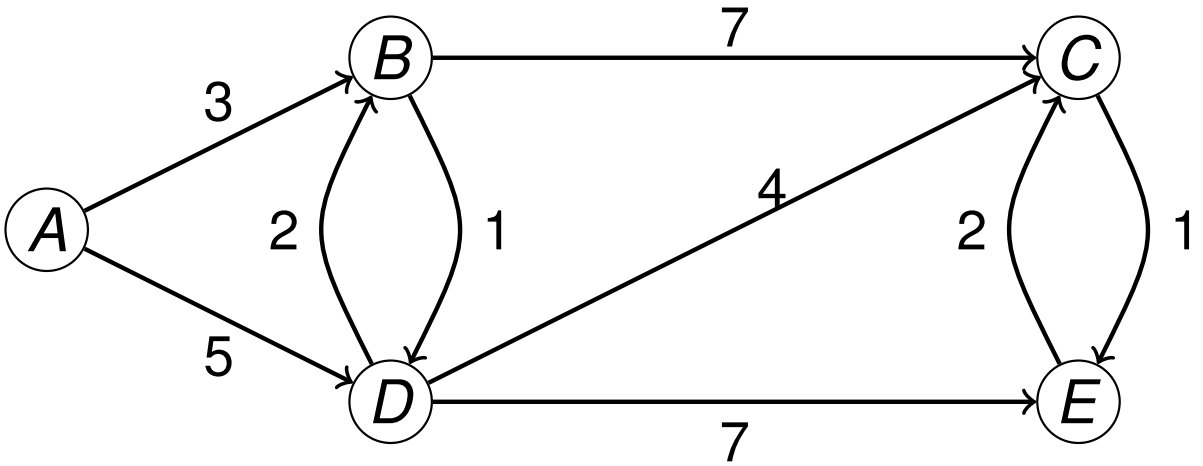


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B\}$ $\quad Q = \{C, D, E\}$

	A	B	C	D	E
d	0	3	∞	5	∞
π	–	A		A	

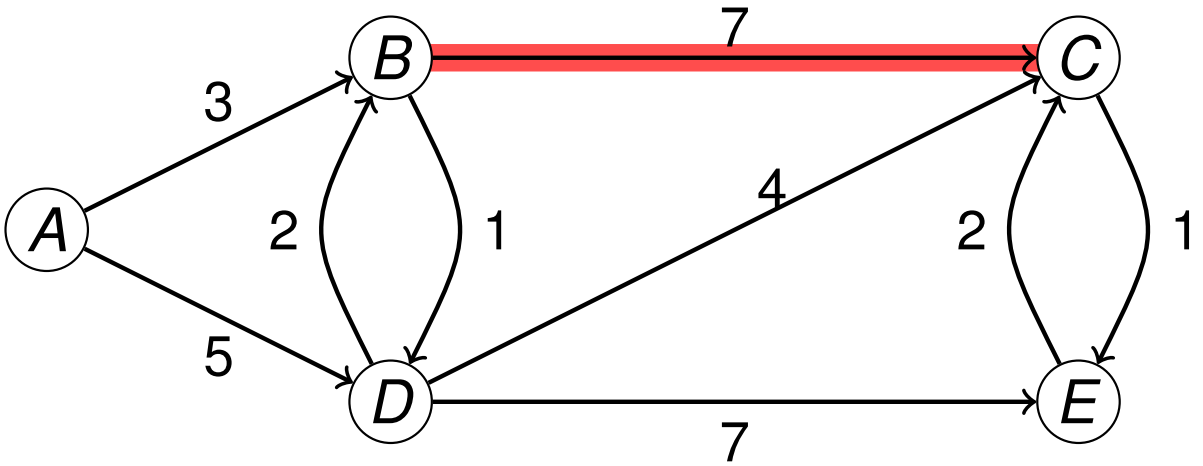


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B\}$ $\quad Q = \{C, D, E\}$

	A	B	C	D	E
d	0	3	∞	5	∞
π	–	A		A	

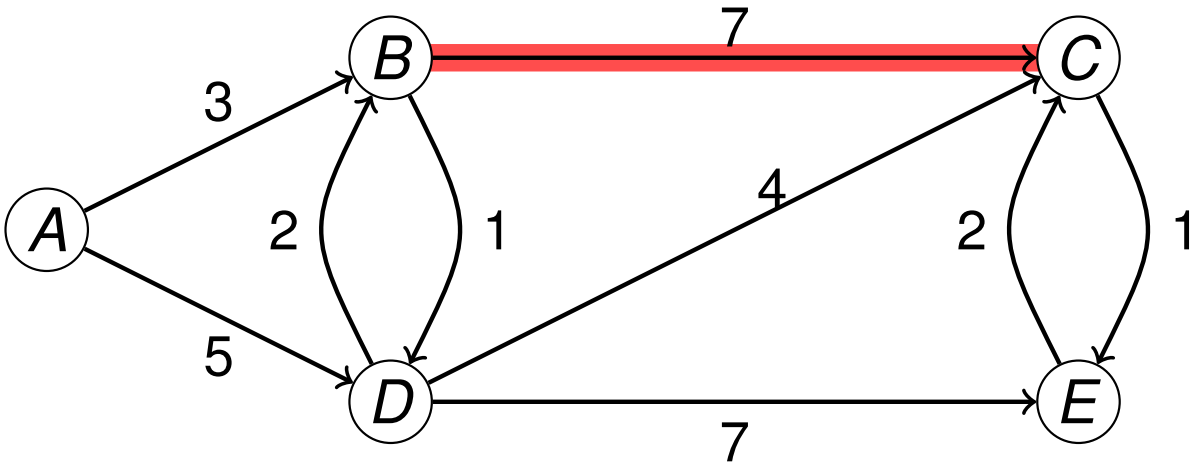


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B\}$ $\quad Q = \{C, D, E\}$

	A	B	C	D	E
d	0	3	10	5	∞
π	$-$	A	B	A	

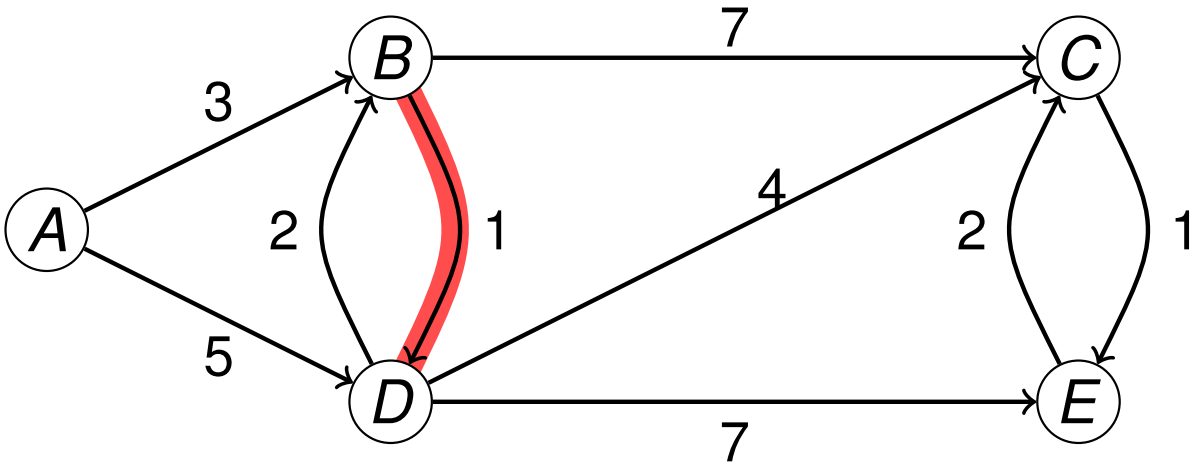


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B\}$ $\quad Q = \{C, D, E\}$

	A	B	C	D	E
d	0	3	10	5	∞
π	–	A	B	A	

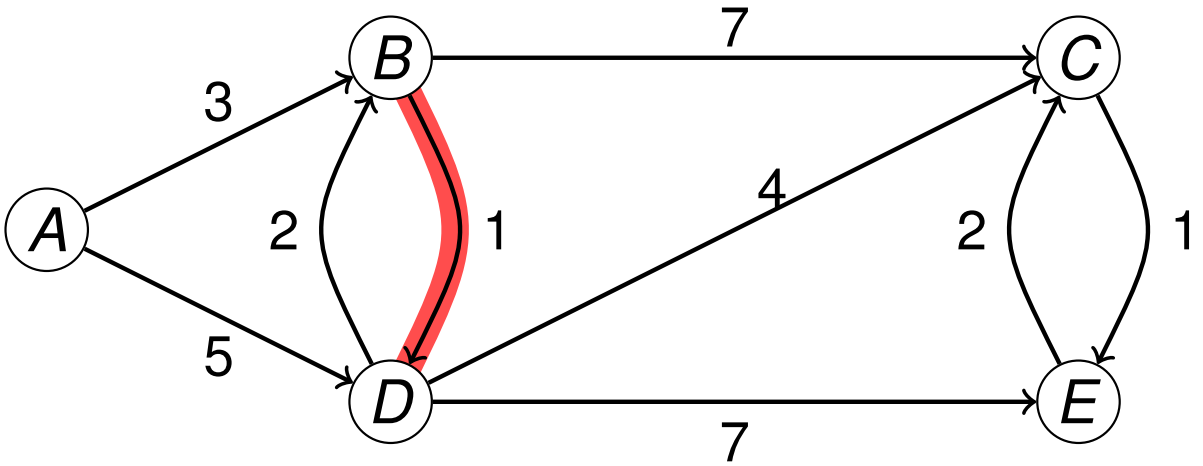


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B\}$ $\quad Q = \{C, D, E\}$

	A	B	C	D	E
d	0	3	10	4	∞
π	–	A	B	B	

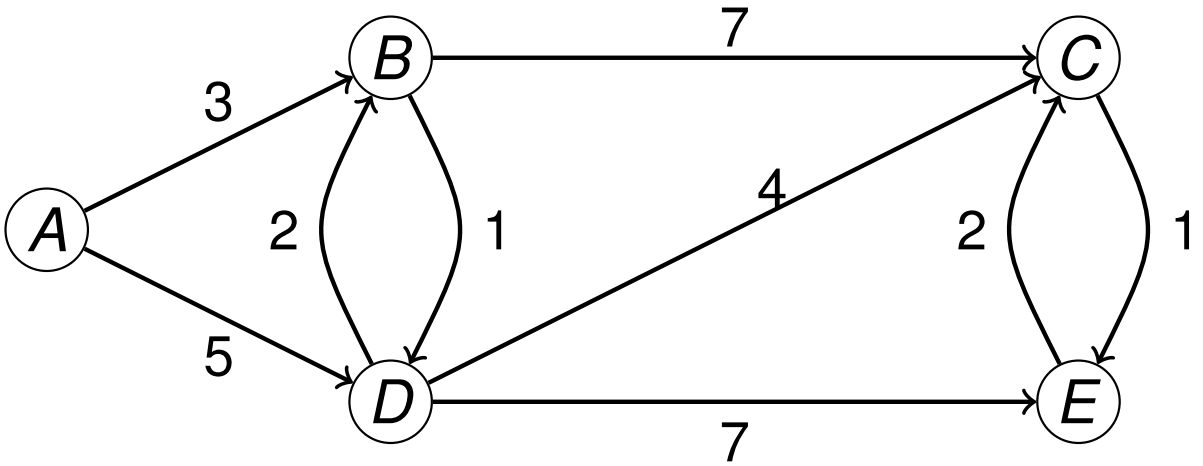


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B, D\}$ $\quad Q = \{C, E\}$

	A	B	C	D	E
d	0	3	10	4	∞
π	–	A	B	B	

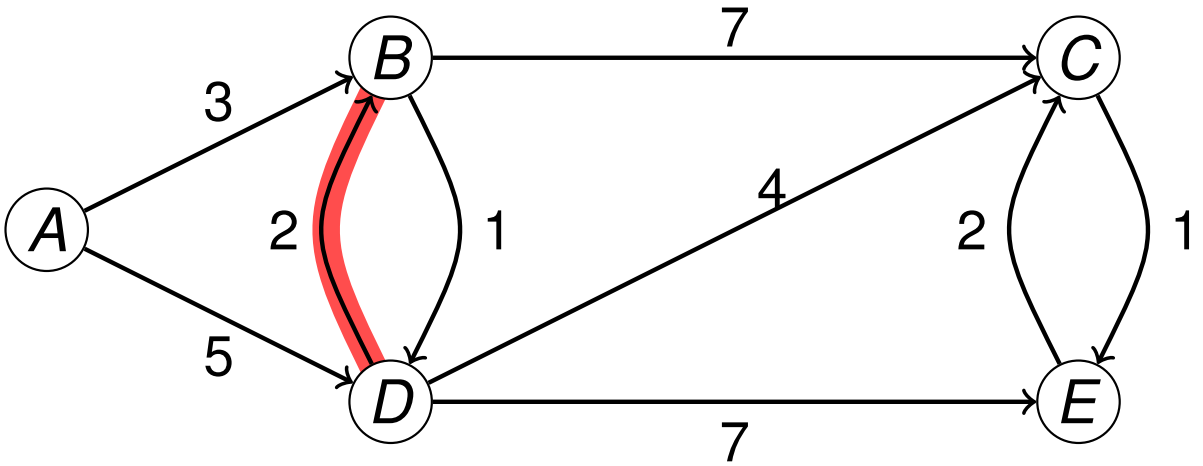


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B, D\}$ $\quad Q = \{C, E\}$

	A	B	C	D	E
d	0	3	10	4	∞
π	–	A	B	B	

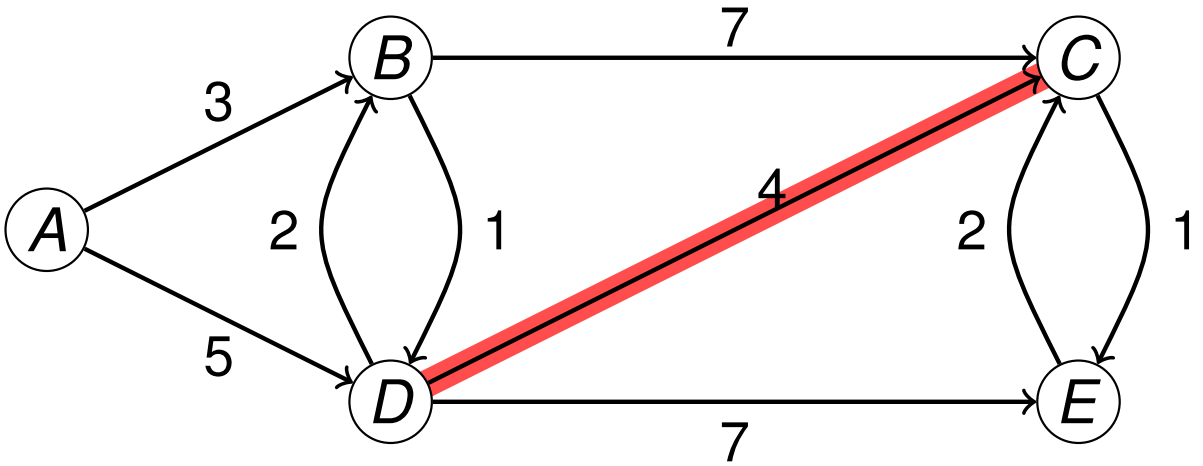


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B, D\}$ $\quad Q = \{C, E\}$

	A	B	C	D	E
d	0	3	10	4	∞
π	–	A	B	B	

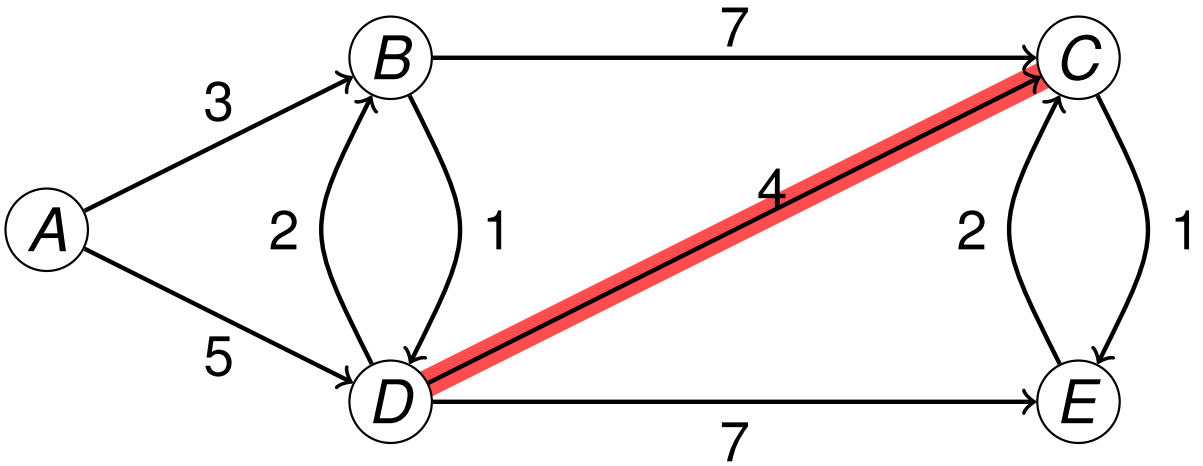


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B, D\}$ $\quad Q = \{C, E\}$

	A	B	C	D	E
d	0	3	8	4	∞
π	–	A	D	B	

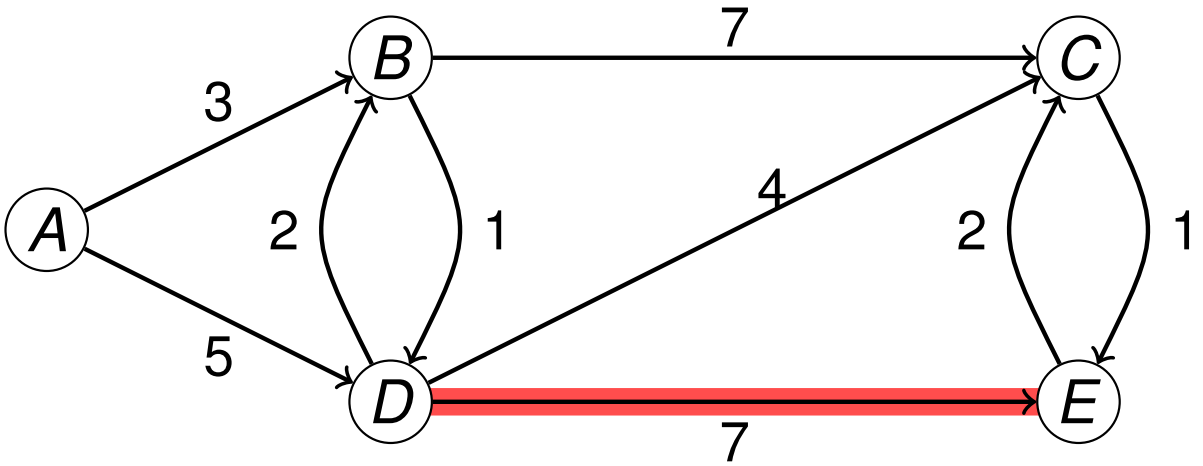


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B, D\}$ $\quad Q = \{C, E\}$

	A	B	C	D	E
d	0	3	8	4	∞
π	–	A	D	B	

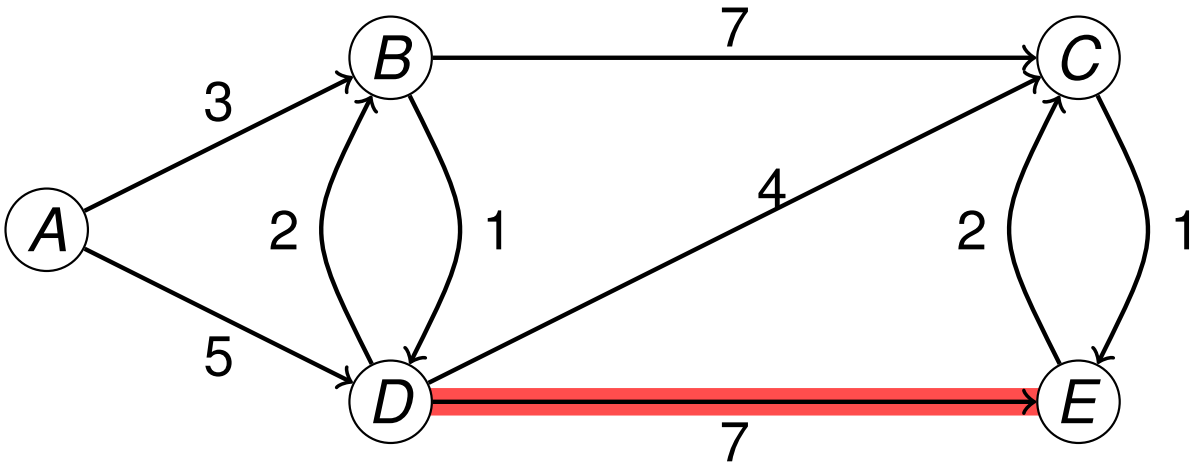


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B, D\}$ $\quad Q = \{C, E\}$

	A	B	C	D	E
d	0	3	8	4	11
π	–	A	D	B	D

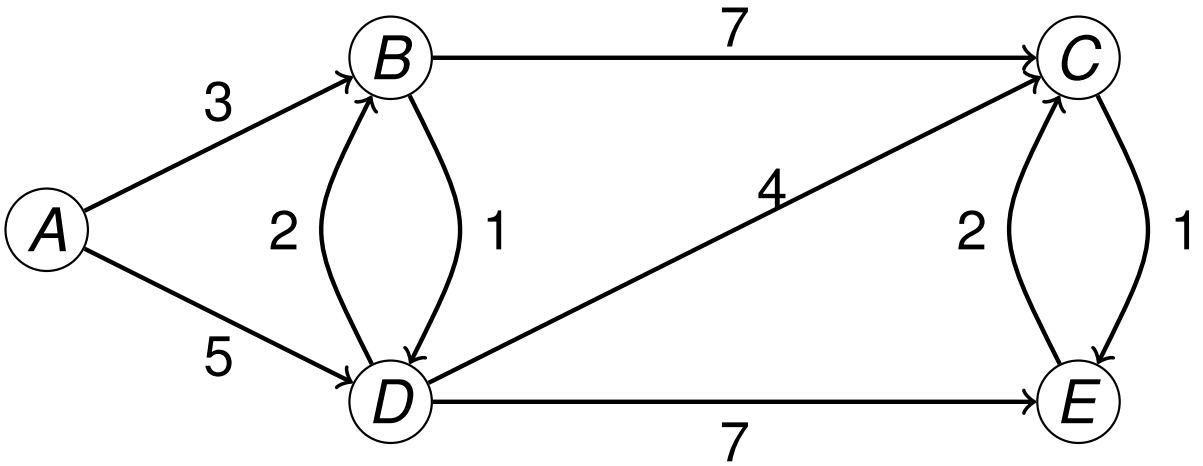


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B, D, C\}$ $\quad Q = \{E\}$

	A	B	C	D	E
d	0	3	8	4	11
π	–	A	D	B	D

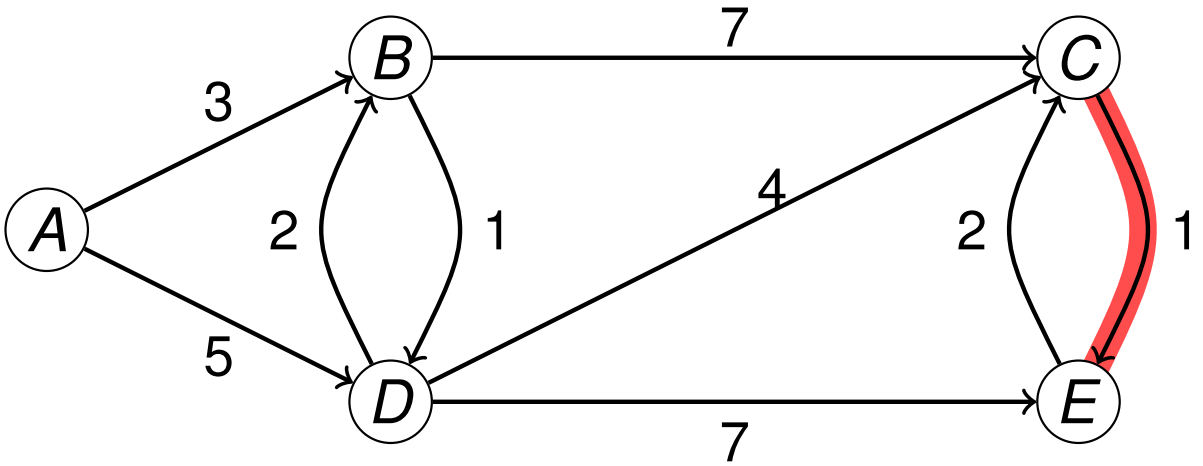


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B, D, C\}$ $\quad Q = \{E\}$

	A	B	C	D	E
d	0	3	8	4	11
π	–	A	D	B	D

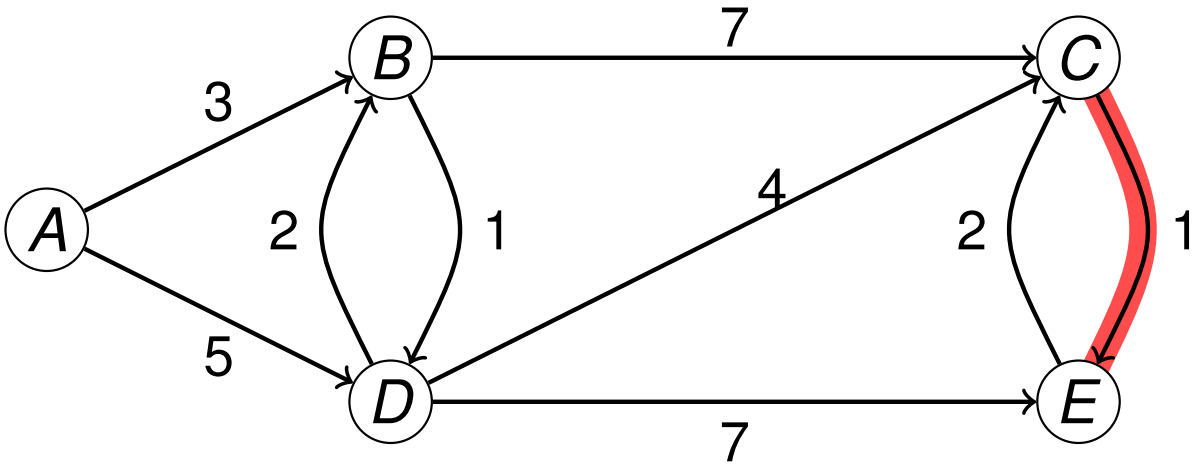


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B, D, C\}$ $\quad Q = \{E\}$

	A	B	C	D	E
d	0	3	8	4	9
π	–	A	D	B	C

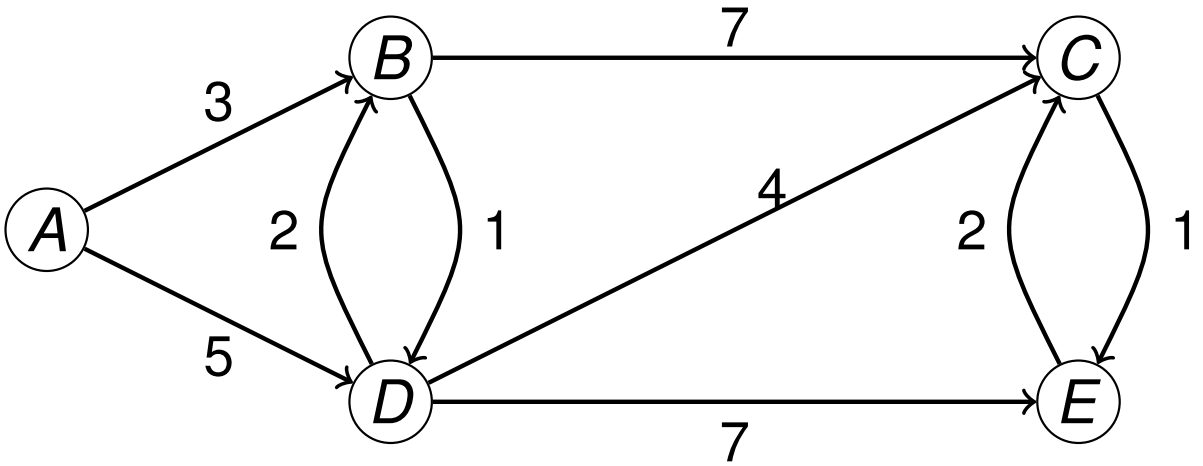


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B, D, C, E\}$ $\quad Q = \emptyset$

	A	B	C	D	E
d	0	3	8	4	9
π	–	A	D	B	C

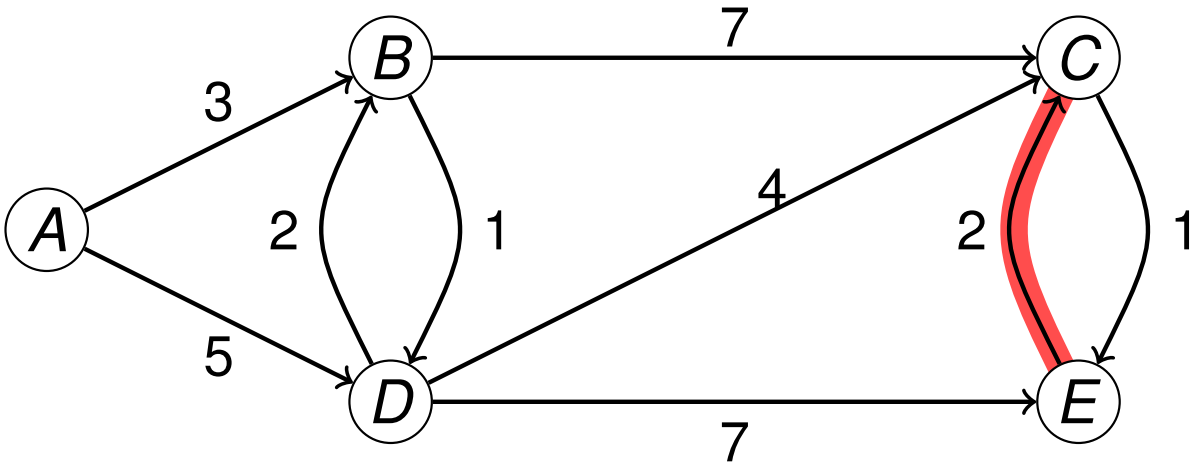


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B, D, C, E\}$ $\quad Q = \emptyset$

	A	B	C	D	E
d	0	3	8	4	9
π	–	A	D	B	C

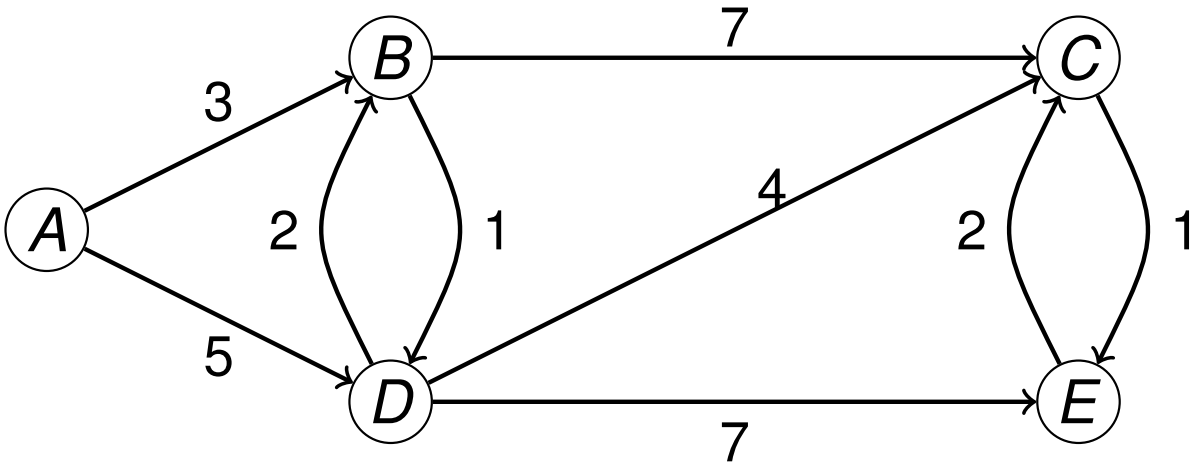


- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

$S = \{A, B, D, C, E\}$ $\quad Q = \emptyset$

	A	B	C	D	E
d	0	3	8	4	9
π	–	A	D	B	C



- Initialization: distance to source A is 0, $S = \emptyset$, $Q = V$.
- S stores the vertices v for which we already found $\delta(A, v)$.
- Q stores all the other vertices.
- while Q is not empty:
  - remove from Q the vertex u for which $d(u)$ is minimum;
  - add to S this vertex u;
  - relax all edges leaving u.

Dijkstra's Algorithm

```
INITIALIZE-SINGLE-SOURCE(G, s)
for each vertex v ∈ V(G) do
    d(v) ← ∞
    π(v) ← NIL
d[s] ← 0
```

```
RELAX(u, v, w)
if d(v) > d(u) + w(u, v) then
    d(v) ← d(u) + w(u, v)
    π(v) ← u
```

```
DIJKSTRA(G, w, s)
INITIALIZE-SINGLE-SOURCE(G, s)
S ← ∅
Q ← V(G)
while Q ≠ ∅ do
    u ← EXTRACT-MIN(Q)
    S ← S ∪ {u}
    for each vertex v ∈ Adj(u) do
        RELAX(u, v, w)
```

Running time of Dijkstra's Algorithm

- Initialization is done in $O(V)$ time.
  - two operations per vertex
- Finding the vertex v in Q with minimum $d(v)$ takes $O(V)$ time and this is done V times.
  - to find the minimum, just scan the set Q
  - to compute the new vertex in S, find the new minimum of Q
- Relaxation takes (in total) $O(E)$ time
  - every edge is relaxed once

$\Longrightarrow$ Total running time is $O(V + V^2 + E) = O(V^2)$

Running time of Dijkstra's Algorithm

- Initialization is done in $O(V)$ time.
  - two operations per vertex
- Finding the vertex v in Q with minimum $d(v)$ takes $O(V)$ time and this is done V times.
  - to find the minimum, just scan the set Q
  - to compute the new vertex in S, find the new minimum of Q
- Relaxation takes (in total) $O(E)$ time
  - every edge is relaxed once

$\Longrightarrow$ Total running time is $O(V + V^2 + E) = O(V^2)$

- However: by using a more sophisticated implementation (for extracting the minimum from Q), Dijkstra's Algorithm runs in $O(V \log V + E)$ time
  - see the text-book for details

Properties of Shortest Paths and Relaxation

Triangle Inequality

For all edges (u, v), we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.

Properties of Shortest Paths and Relaxation

Triangle Inequality

For all edges (u, v), we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.

Optimal Substructure

Any subpath of a shortest path is also a shortest path.

Properties of Shortest Paths and Relaxation

Triangle Inequality

For all edges (u, v), we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.

Optimal Substructure

Any subpath of a shortest path is also a shortest path.

Upper Bound Property

For every vertex v, we always have $d(v) \geq \delta(s, v)$.

Properties of Shortest Paths and Relaxation

Triangle Inequality

For all edges (u, v), we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.

Optimal Substructure

Any subpath of a shortest path is also a shortest path.

Upper Bound Property

For every vertex v, we always have $d(v) \geq \delta(s, v)$.

No-path Property

If $\delta(s, v) = \infty$, then we have $d(v) = \infty$ at every iteration.

Properties of Shortest Paths and Relaxation

Triangle Inequality

For all edges (u, v), we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.

Optimal Substructure

Any subpath of a shortest path is also a shortest path.

Upper Bound Property

For every vertex v, we always have $d(v) \geq \delta(s, v)$.

No-path Property

If $\delta(s, v) = \infty$, then we have $d(v) = \infty$ at every iteration.

Convergence Property

If there is a shortest path from s to v including the edge (u, v) and if $d(u) = \delta(s, u)$, then we obtain $d(v) = \delta(s, v)$ when (u, v) is relaxed.

Correctness of Dijkstra's Algorithm

We need to prove that the Loop Invariant always remains true:

At the start of each iteration of the while loop, $d(v) = \delta(s, v)$ for every $v \in S$.

- Initialization: at the start of the algorithm S is empty, so the loop invariant is trivially true.
- Maintenance: we need to show that $d(u) = \delta(s, u)$, when u is added to S.
- Termination: at the end, S contains every vertex, which implies that $d(v) = \delta(s, v)$ for all vertices v in the graph.